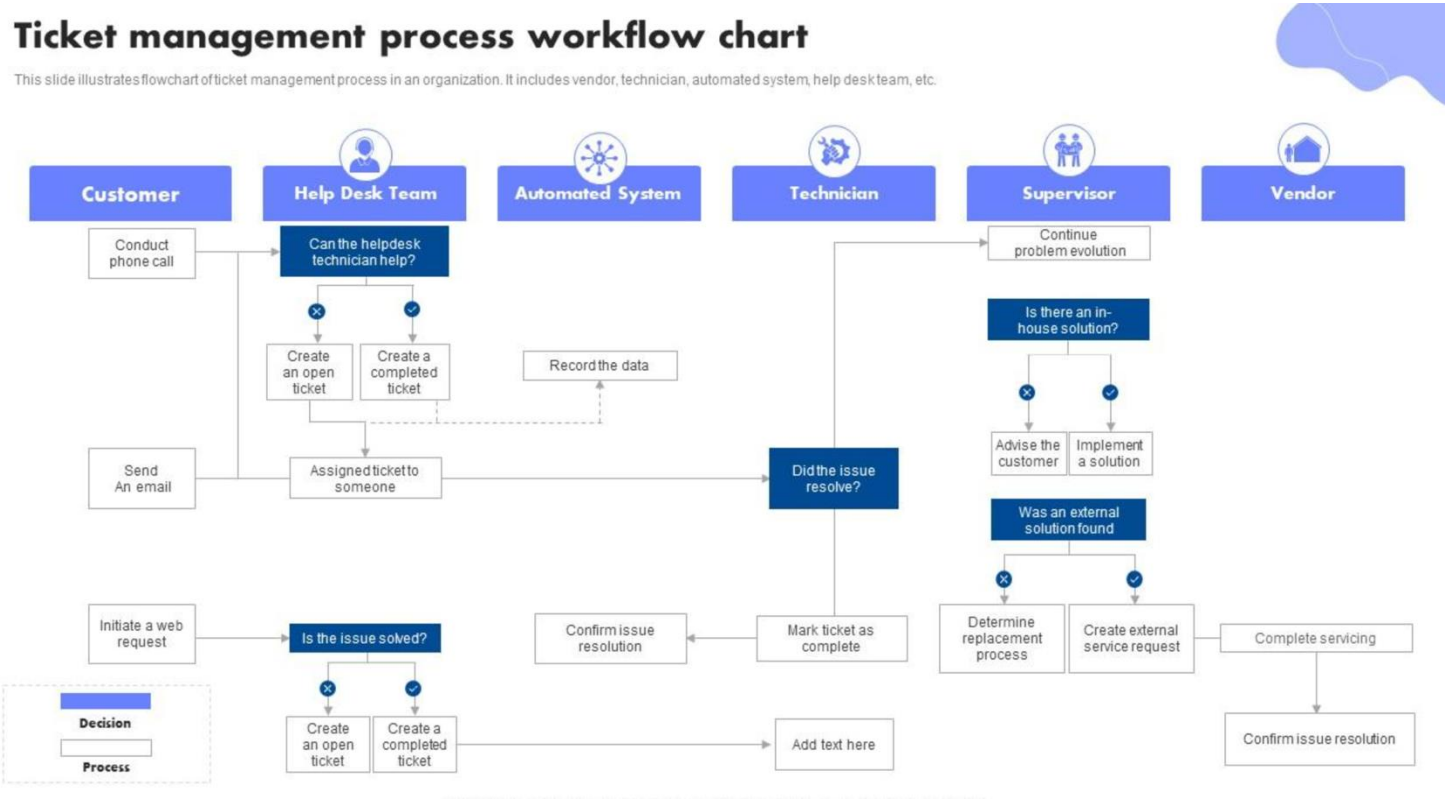
AutoResolve: Enterprise RAG Agent for IT Support

Automating Tier-1 Triage and Resolution

Shaury Pratap Singh | Master of Science in Data Science | New Jersey Institute of Technology

The IT Support Bottleneck

- **High Volume:** As enterprise support scales, manual ticket triage becomes a critical bottleneck.
- **Linguistic Chaos:** Human language is unstructured, containing typographical errors, sarcasm, and grammatical complexity.
- **Inefficiency:** Human-in-the-loop triage drives up Mean Time to Resolution (MTTR) and operational costs.



The Objective

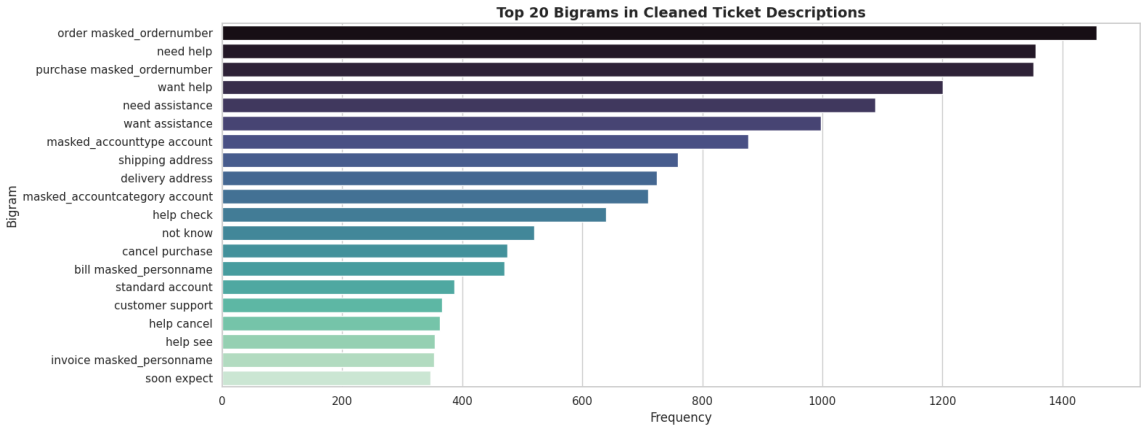
Goal 1: Build a high-accuracy intent classifier to distinguish between 27 distinct customer support categories.

Goal 2: Advance beyond simple classification into an active Large Language Model (LLM) Agent.

Goal 3: Retrieve exact enterprise policies and generate hallucination-free resolutions in real-time.

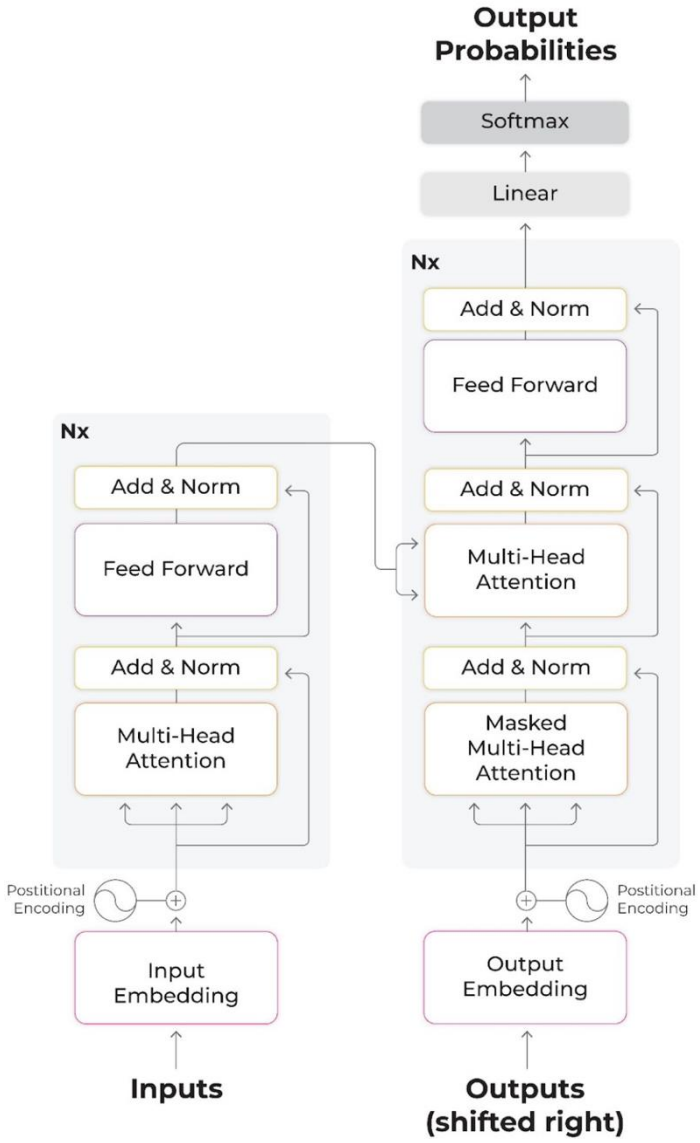
Data & Linguistic Engineering

- Utilized the Bitext Customer Support dataset containing 26,872 labeled utterances.
- **The Challenge:** Standard tokenization destroys masked variables (e.g., {{Order Number}}).
- **The Fix:** Engineered a custom Regex pipeline to isolate and protect placeholders as single tokens.
- Applied spaCy lemmatization and removed stop words to normalize linguistic expressions



Baseline vs. Deep Learning

- **The Baseline:** Term Frequency-Inverse Document Frequency (TF-IDF) combined with a Support Vector Machine (SVM) achieved 99.00% accuracy.
- **The Limitation:** TF-IDF evaluates words in isolation and ignores syntactic order. It struggles with lexical ambiguity.
- **The Upgrade:** Deployed DistilBERT to leverage bidirectional self-attention mechanisms.

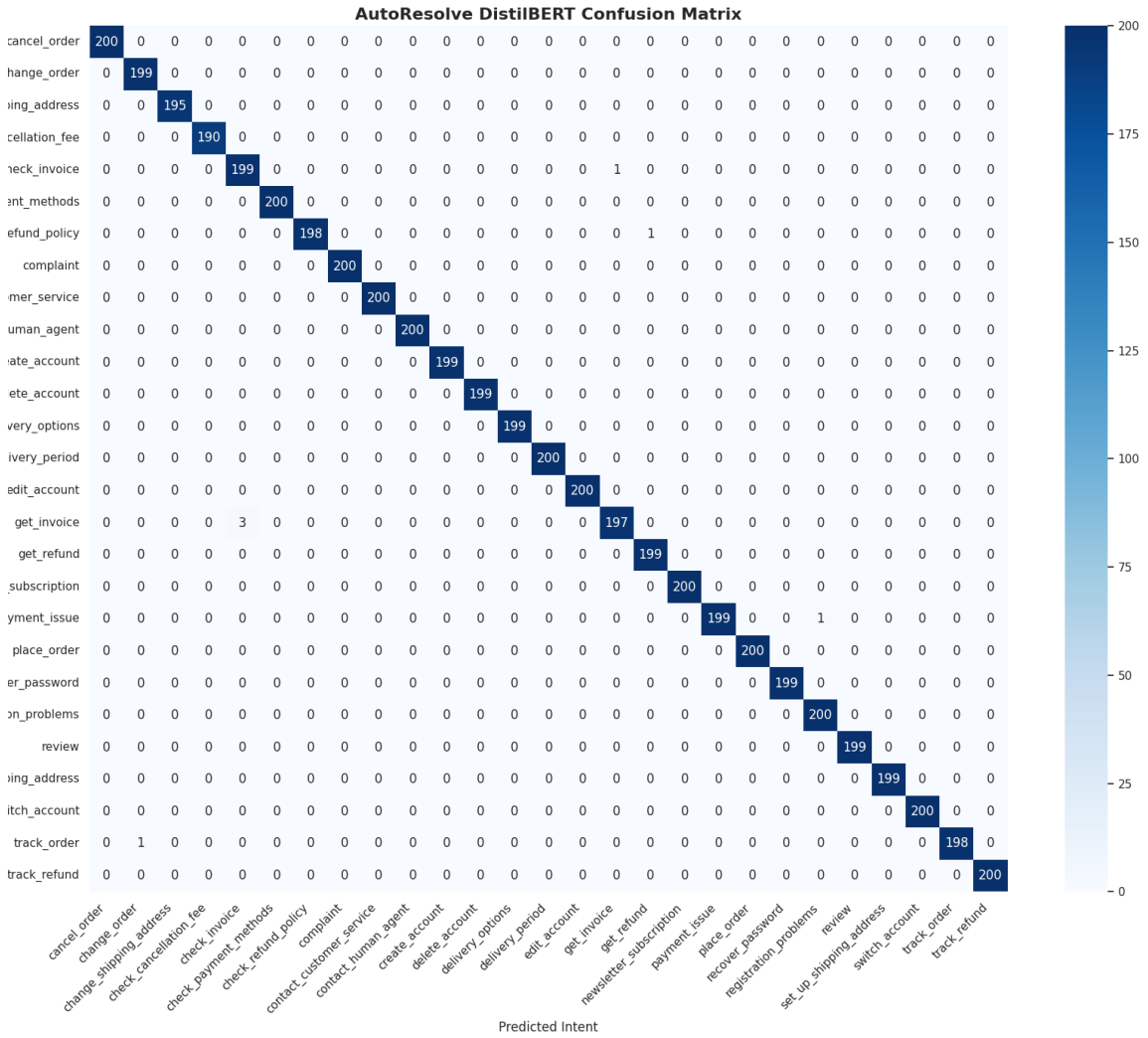


Intent Classification Results

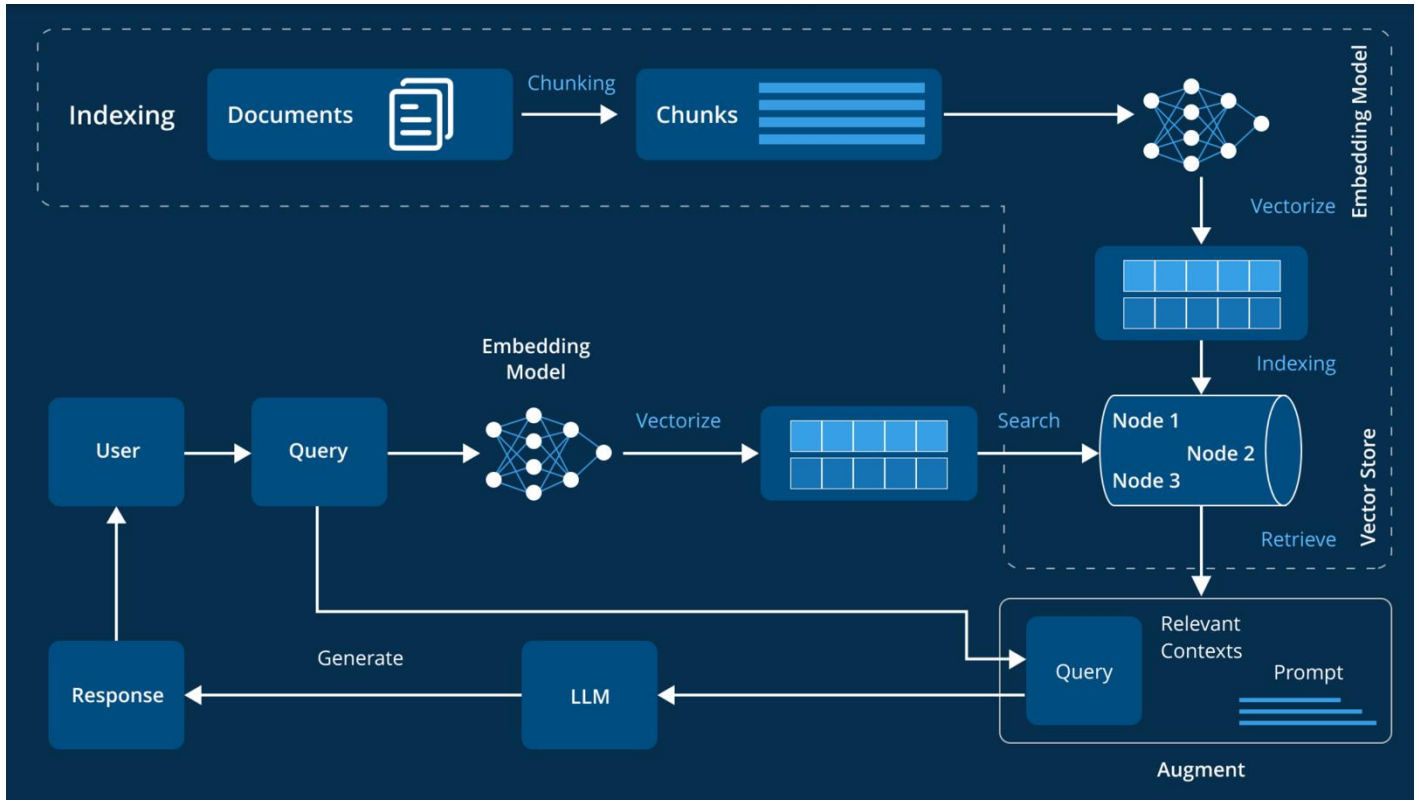
DistilBERT achieved an exceptional macro F1-score of 99.81%.

Model actively learned semantic boundaries across all 27 classes without majority-class bias.

Effectively captured contextual nuance and handled out-of-vocabulary typographical errors.



RAG: Policy-Aware Retrieval



Classification alone doesn't solve the user's problem. We need an answer.

Built a Retrieval-Augmented Generation (RAG) pipeline to ground the AI in factual data.

Vector Database: Indexed an internal knowledge base using FAISS (Facebook AI Similarity Search).

Matching: Converts user queries into embeddings and retrieves the top policy documents via Cosine Similarity.

Generative Resolution & Optimization

Injected the retrieved FAISS context into a strict system prompt instructing the LLM to act as a Tier 1 agent.

Constrained the model strictly to the retrieved policy to prevent hallucinations.

Deployed a highly efficient Generative Large Language Model (Qwen2.5-0.5B).

Engineered to run inference concurrently on standard cloud CPU environments, removing the need for massive GPU clusters.

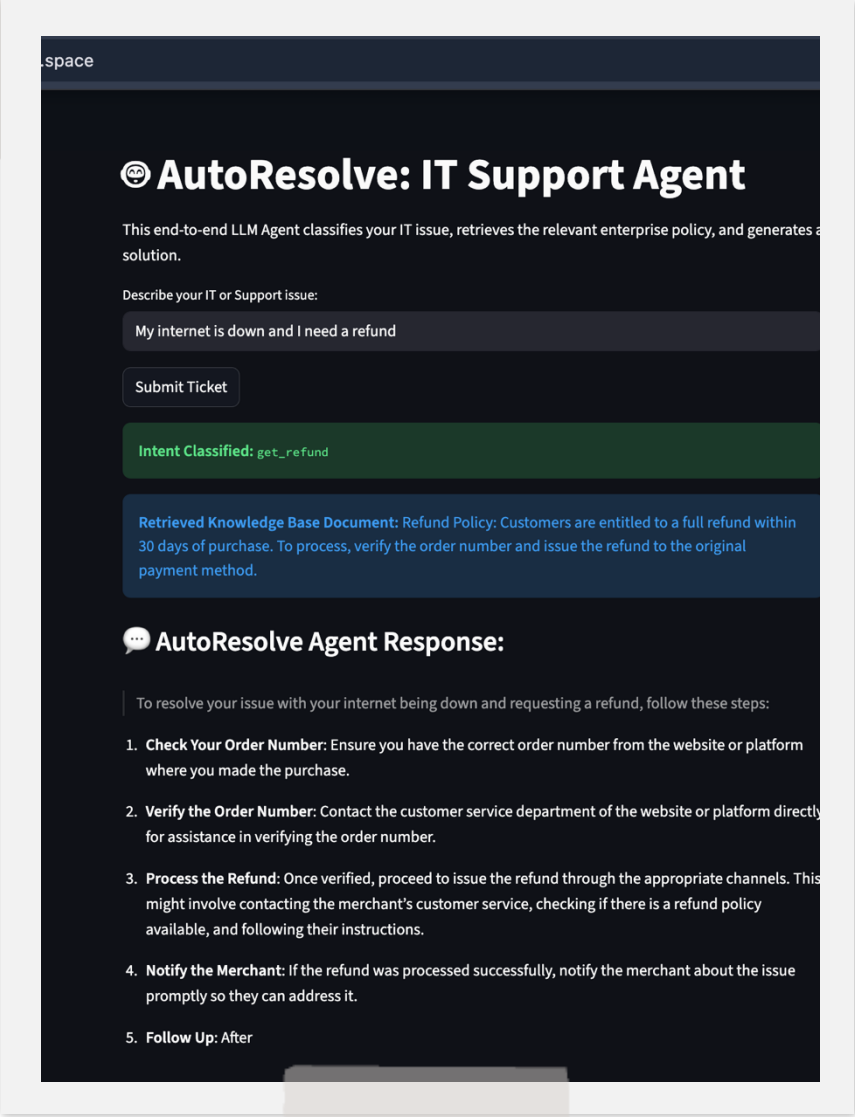
Limitations & Future Roadmap

Current Limitations:

- **Lexical Ambiguity:** The model occasionally struggles to differentiate between informational policy queries and transactional requests (e.g., *asking* about a refund vs. *demanding* a refund).
- **Out-of-Vocabulary (OOV) Degradation:** Severe typographical errors can destroy semantic verb anchors (e.g., typing "Icate" instead of "Locate"), forcing the model to guess based on nouns.

Future Work:

- **Multi-Modal RAG Expansion:** Upgrading the FAISS vector database to ingest and retrieve non-text enterprise documents, such as PDF manuals, flowcharts, and images.
- **RLHF Implementation:** Integrating Reinforcement Learning from Human Feedback to reduce classification degradation and continuously align the agent's tone with enterprise brand guidelines.



Conclusion & Live Demo

Successfully transitioned static categorization into dynamic, context-aware automated IT resolution.

Demonstrates clear business value by automating Tier-1 workflows and reducing computational overhead.

Live App: huggingface.co/spaces/Shauryaaa05/AutoResolve-Agent

Codebase: github.com/shaury05/AutoResolve-NLP-Triage-RAG-Support-Agent